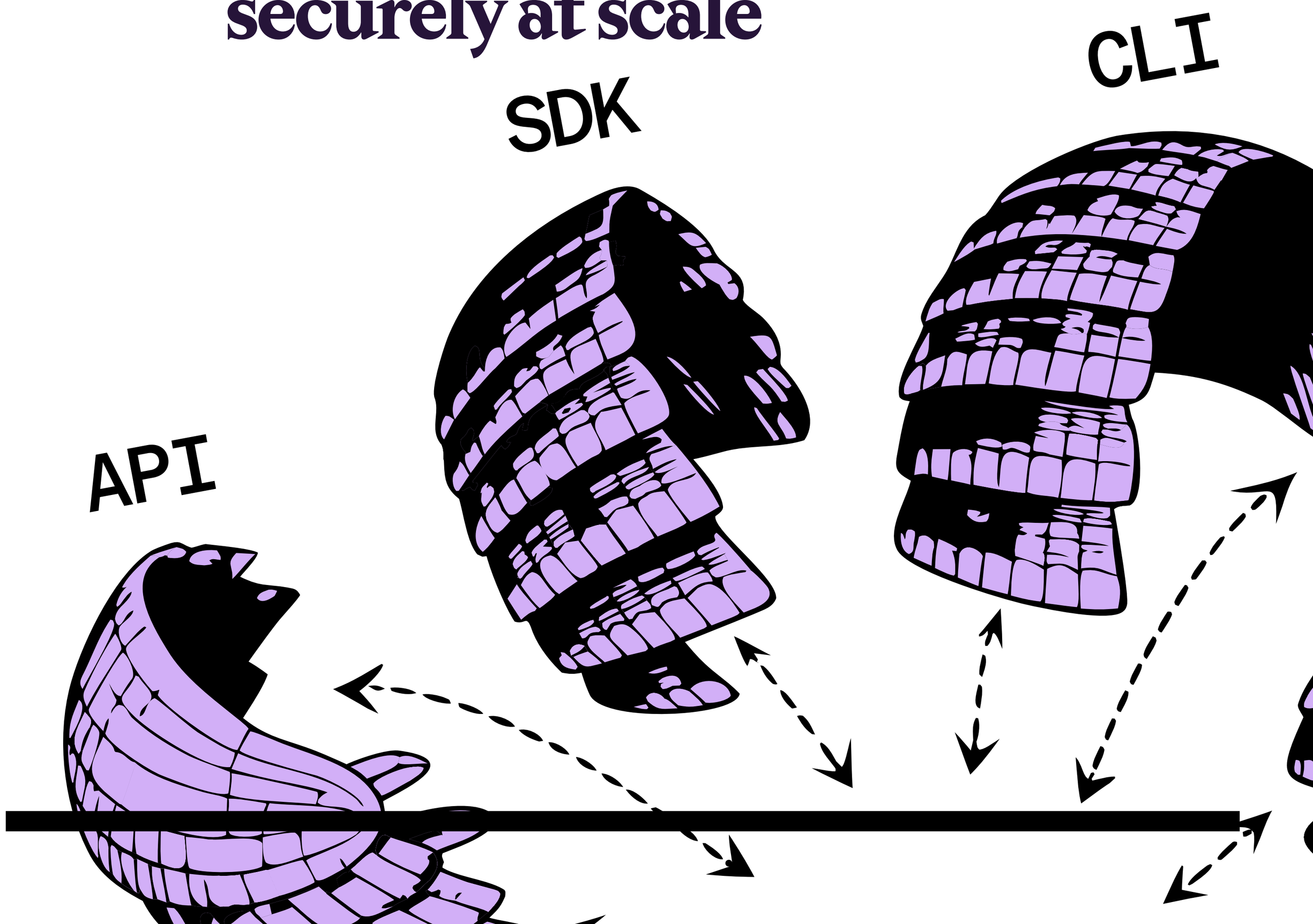
2026 EDITION

Oz by warp

Cloud Agents for Enterprise

The ultimate guide to orchestrating agents securely at scale



AI Tools Everywhere, Orchestration Nowhere

Contents

2 The Core Problem
4 Why Orchestration Matters Now
7 What Teams Automate with Oz
13 Success with Cloud Agents
16 Your First Automated Workflow

The Enterprise AI Gap

Your engineers use AI agents. They're faster at writing code. But something else is happening: visibility is down. You have no audit trail. No governance. No way to prevent duplicate efforts or security exposure.

This isn't a failure of AI adoption—it's a failure of orchestration. AI still requires an engineer sitting at a keyboard, one task at a time. The tools accelerate individual work, but they don't work independently, and they don't compound across your team.

Industry research tells the story: most organizations have adopted AI coding tools, but fewer than one in ten have deployed agents in production.[1] Analysts predict 90% of enterprise software engineers will use AI code assistants by 2028, and 40% of enterprise applications will feature AI agents by 2026.[2] The gap isn't adoption intent. It's the infrastructure to deploy agents across your organization—with the governance, visibility, and automation that enterprise teams require.

The Core Problem

Most AI coding tools work alongside individual engineers—helping them move faster on whatever task is in front of them. They still require a human in the loop for every action. However, enterprises need more than individual acceleration. Your codebase spans dozens of repos. Your workflows span teams. Your compliance requirements span everything.

The cost of waiting for orchestration compounds daily. On the technical side, unmanaged agents generate changes without audit trails, making it impossible to track what was AI-generated and why. Organizationally, teams stall AI adoption because there's no clear path forward. And from a security perspective, "shadow AI" spreads as engineers adopt personal tools without governance, creating exposure you can't see until it's a problem.

What enterprises actually need is a way to scale AI agents from individual productivity tools to team-wide infrastructure—with visibility into what agents are doing, governance over how they operate, and the ability to run them autonomously in the background. When agents can work independently, engineers can focus on shipping features and building better products.

[1], [2] Recon Analytics, Enterprise AI Adoption Survey, March 2025–January 2026, n=120,000+ enterprise respondents.

Introducing Oz: the orchestration platform for cloud agents

Oz is Warp's programmable agent platform for running and coordinating AI agents at scale.

Deploy agents that work autonomously in the cloud—scheduled, triggered, or launched on demand—with full visibility, enterprise controls, and seamless handoff to engineers when human judgment is needed. Every agent is auto-tracked: producing a link, audit trail, and CLI/API control.

The Core Difference

Most cloud agents run in single-repo environments. Oz is the programmable agent stack—combining a best-in-class agent with built-in orchestration that makes agents work across repos, across teams, and with enterprise security controls. It's the difference between giving engineers a faster bicycle and giving the organization a transportation system.

Capability	Single-Engineer AI Tools	Oz
Codebase scope	Single repo only	Coordinated changes across multiple repositories
Session transparency	Black box execution	Every PR links to the agent session
Recoverability	Start over locally	Pick up where the agent left off—debug, correct, continue
Team-wide automation	Per-developer tool	One person configures, team benefits
Trigger & Integrations	Manual invocation	Built-in: cron, API, Slack, Linear, Github events
Auditability	Limited visibility	Full visibility: every action, every run, every decision
Concurrent Execution	One agent at a time	Run dozens of agents in parallel—escape laptop limits

Oz is built for organizations where security isn't optional. With self-hosting, environment compute and code stay within your security boundary. Every change is logged, and humans review every change before it ships.

Why Orchestration Matters Now

"Single agents are not the bottleneck. Orchestration is."

The question isn't whether AI agents can help individual engineers—most teams have seen that firsthand. The question is how you scale agentic development off individual laptops and into team-wide infrastructure: agents that run in the background, work across multiple repositories, and operate within your security and compliance requirements.

What's possible when orchestration is solved: At Warp, 60% of merged PRs are created by an agent—agents removing stale feature flags, implementing bug fixes, making changes from Slack based on ideas, and conducting product research. This isn't theory. It's the daily reality of teams with the right orchestration layer.

The industry data reinforces the urgency. The majority of employees already use personal, unmanaged AI tools. Most organizations lack comprehensive AI controls, and many have no clear owner of AI risk. Perhaps most telling: Forrester predicts that 75% of companies attempting to build their own agentic systems will fail.[3]

The question you'll need to answer: "How are we using AI to ship faster than competitors?"

Oz gives you a defensible answer—team-wide automation with governance, not just individual productivity tools you can't measure or control. And unlike DIY orchestration, you're not trading engineering capacity for infrastructure.

[3] Forrester Research, Predictions 2025: An AI Reality Check Paves The Path For Long-Term Success, 2024.

The Agent-Native Software Development Lifecycle

This is the Agent-Native SDLC: a continuous loop where cloud agents work asynchronously at every stage—from requirements through implementation, deployment, and troubleshooting—while engineers review, steer, and focus on the work that requires human judgment.

01 Plan

WEEKLY USER FEEDBACK DIGEST
Summarize customer feedback across channels and share a Monday morning roundup.
- SCHEDULER: 8 AM MONDAY MORNINGS
- SLACK MESSAGE

TICKET TRIAGE AND PRIORITIZATION
Group Linear tickets, propose owners and priority, and draft a short plan for each epic.
- LINEAR INTEGRATION
- ARTIFACT: PLAN

PRD OUTLINE FROM NEW REQUESTS
Automatically turn inbound asks into scope, success criteria, and open questions the team can align on.
- ARTIFACT: PRD DRAFT
- ARTIFACT: PLAN

02 Prototype

PROTOTYPE END-TO-END FLOWS IN AN ISOLATED SANDBOX
Explore multiple approaches without touching mainline code
- SESSION LINKS
- ITERATION COMMITS
- PROTOTYPE BRANCHES

PROTOTYPE MULTIPLE APPROACHES IN PARALLEL AND COMPARE TRADEOFFS
Build two or three versions (e.g., different architectures or libraries) and deliver a side-by-side evaluation
- REFACTOR COMMITS
- CHECK RUNS

REALISTIC SAMPLE DATA AND FIXTURES
Seed the sandbox with representative data and edge-case fixtures so the prototype behaves like production.
- SEED SCRIPT
- FIXTURES
- SAMPLE DATA

03 Build

TICKET TO PR IMPLEMENTATION
Implement end-to-end on a clean branch and open a reviewable PR with a clear summary.
- TICKET BRANCH
- PR DRAFT

SAFE REFACTORS WITH VALIDATION
Apply large mechanical refactors, run checks, and document risk and follow-ups in the PR
- PRS

COORDINATED MULTI-REPO CHANGES
Make consistent updates across repos and open linked PRs for review
- MULTI-REPO CHANGES
- PRS

04 Review + Merge

FIRST-PASS CODE REVIEW
Summarize intent, flag risks, and suggest improvements before humans dive in.
- PR REVIEW COMMENTS
- SUGGESTED CHANGES

ADDRESS REVIEW FEEDBACK
Automatically address requested changes and respond with clear rationale.
- ITERATION COMMITS

SECURITY AND DEPENDENCY SCAN
Check for vulnerabilities and risky dependencies and attach a short remediation plan.
- SECURITY SCANS
- REMEDIATION PLAN

05 Validate

TESTS AND CI CONFIDENCE
Generate or update tests, run the suite, and iterate until everything is green.
- TEST GENERATION
- CHECK RUNS

END-TO-END UI VERIFICATION
Use computer use to validate key flows and produce a QA checklist with evidence
- COMPUTER USE
- QA CHECKLIST

BUG REPRODUCTION AND VALIDATION
Test the changes against the original bug report
- COMPUTER USE
- GITHUB ISSUE

06 Deploy + Monitor

RELEASE NOTES AND ROLLOUT SUMMARY
Generate release notes and a rollout recap that teams can reuse.
- RELEASE NOTES
- ROLLOUT SUMMARY

REGRESSION MONITORING
Watch dashboards and alerts post-deploy and flag regressions early.
- DASHBOARD WATCH
- REGRESSION ALERTS

INCIDENT TRIAGE AND MITIGATION
Collects logs and deploy context, then recommend a rollback or hotfix plan.
- INCIDENT CONTEXT
- HOTFIX PLAN

How Oz Works in Practice

Built for Your Security Requirements

Before we talk about what Oz can do, let's address what's actually blocking you from adopting it. Security and governance concerns are the reason most enterprises haven't moved beyond individual AI tools—and rightfully so. Here's how Oz is built for that reality.

Human approval by default. Out of the box, agents propose changes—they can't ship them. Every change requires human approval before it hits production. If something goes wrong, you have full recoverability: see exactly what the agent did and correct course before anything deploys.

Environments are fully isolated. Agents can only access what's explicitly configured—nothing more. Agents never inherit permissions they weren't explicitly granted.

Every action is logged. Every session is recorded. Every PR links back to the full conversation. Team members can open the session, see exactly what the agent saw and decided, and fork it to continue or correct the work.

Keep code within your security boundary. Many enterprises have strict constraints: no code leaves the internal network, no external services can access production systems. Oz supports this. With self-hosting, environment compute runs on your infrastructure, subject to your access controls. Warp handles orchestration; your code never leaves.

Warp is SOC 2 compliant with Zero Data Retention agreements with LLM providers. Enterprise customers can bring their own API keys. Every run, every decision, every PR created is captured in a full audit trail.

On the governance side, admins control what agents can do without approval, which models teams can use, and how credentials are shared. Run history is fully observable for cost tracking and attribution—you'll know exactly what's being spent, by whom, and on what.

What Teams Automate with Oz

From Alert to Fix—Without the Investigation Loop

"Waiting for CI/CD failures—or worse, production incidents—is where engineering time disappears."

When CI fails or an alert fires, an Oz agent analyzes logs across multiple services, reproduces the issue, identifies root cause, and proposes a fix with a PR. What used to take 2-4 hours of manual investigation now takes 15 minutes of review.

Example

One engineer built an on-call skill that takes just an alert name and automatically looks up the triggering code, pulls logs via Elastic, queries k8s pods and queues, and lists mitigation steps with executable commands.

This is what single-engineer tools can't do: When a bug spans your shared library and three downstream services, the agent opens coordinated PRs across all four repos—linked to a single session you can review in batch.

The impact

Reduced MTTR. Senior engineers spend time on architecture, not investigation loops. On-call burden drops.

Tech Debt That Actually Gets Cleaned Up

"Tech debt accumulates faster than we can clean it."

	Identify	Modify	Test	PRs	Review	Time
Before	Manual	Sequential	Manual	Fragmented	One-by-one	Days
After (Oz)	Automated	Parallel	Automated	Linked	Batch	Hours
Impact	Focus regained	Less risk	Higher confidence	Cleaner flow	Faster decisions	More feature work

Documentation That Updates Itself

"Documentation is always out of date—and no one has time to fix it."

An Oz agent watches PRs, updates READMEs and docs to match code changes, and creates PRs for review. It runs continuously in the background—no engineer has to remember to do it.

The impact

Onboarding accelerates. New engineers ramp faster. Institutional knowledge stops being locked in the heads of people who might leave.

I'm probably 60%-80% more productive.

I feel like I'm the conductor of an orchestra of junior engineer agents.

Staff Software Engineer, IT Software Co

Case Study: From PR Feedback to Team-Wide Automation

A large enterprise software company expanded agentic development from a small set of AI power users to repeatable workflows across their engineering organization.

They started with two high-leverage loops: a documentation bot that keeps internal docs in sync with code changes, and a testing bot that proposes business-critical tests with label-based triggers and admin controls. Next, they standardized PR comment remediation as a two-step workflow: Oz agents generate the initial review and proposed changes, then a separate agent pulls inline feedback, applies fixes comment-by-comment, runs validation, and optionally creates commits incrementally—so engineers stay in control.

Early outcomes: The testing agent consistently produced actionable feedback in sensitive areas, helping teams catch gaps earlier. The documentation bot kept reference docs aligned across multiple repos, making it easier for engineers to ramp quickly. Together, these workflows form an end-to-end loop: agents generate high-quality proposals, engineers review with full context, and improvements compound across repos.

By the numbers

Faster incident response

1-3 hours

saved on debugging

Time saved

90%

less time required context switching

High-quality agent output

95%

agent generated code diffs accepted

“Oz agents maintain our docs autonomously. New engineers onboard faster because the docs are current. On-call engineers go from generic alert to remediation plan in minutes. We went from stale, thin documentation to ever-living documentation that serves both humans and agents”

Staff Software Engineer, IT Software Co

Fits Your Existing Workflow

The use cases above only matter if your team actually adopts the tool.

The question every CTO asks: "Will my team actually use this?"

The answer depends on friction and value. Oz is designed to add zero friction to existing workflows while delivering immediate value—and when engineers do work directly with agents, they're using Warp, a terminal and IDE built for how developers actually want to work.

Core Integrations

- **GitHub:** Agents create PRs, comment on issues, trigger on PR events
- **Slack:** Notifications, approvals, status updates
- **Linear:** Agents create/update tickets, link PRs to issues
- **CLI & API:** Trigger agents from CI/CD, dashboards, or existing automation—or build custom orchestration logic with Oz's SDK

Beyond configuration—full programmability. Oz isn't just a tool you configure; it's a platform you can build on. Use the API or SDK to create custom automations: issue triagers, alert responders, internal tools with agent intelligence built in. If you can describe the trigger and the task, you can automate it.

Multi-model by design. Oz supports multiple LLM providers, so you can choose the right model per workflow—faster and cheaper for routine cleanup, more capable for complex work. No vendor lock-in: switch models when priorities, pricing, or reliability change.

Why this matters: Engineers still review PRs, use GitHub, check Slack—no new tools to learn. The agent infrastructure is invisible to them. What they see: work getting done overnight, PRs ready for review in the morning, tech debt actually getting cleaned up, and the backlog shrinking instead of growing.

The Warp Advantage: Built for Agentic Development

Agents won't get everything right—and that's expected. The question is what happens when engineers need to step in. With Warp, they stay in the tools they already use: a terminal and IDE built for agentic development. No context-switching to a separate dashboard or unfamiliar interface.

With most AI tools, intervention means starting over: re-reading the code, re-understanding the context, re-doing the work the agent already attempted. That friction adds up. It's the hidden tax on every AI workflow.

Warp eliminates that friction. Engineers can see exactly what the agent did, pick up where it left off, and course-correct without losing context. The session is transparent, forkable, and continues seamlessly between cloud and local. Every PR links back to the full agent session—team members can review the reasoning, fork to explore alternatives, or continue locally.

Steer mid-run, don't restart. When requirements shift or scope needs adjustment, engineers can guide the agent while it's working—"don't modify public APIs," "focus on the backend first"—without canceling and starting over. Human oversight becomes a lightweight checkpoint, not a workflow reset.

Trusted by nearly a million engineers at leading companies:

GitHub DOORDASH vmware

amazon NETFLIX docker Retool

Success with Cloud Agents

You've seen what Oz does and how it stays secure. Now the question is: how do you get there?

Build vs. Buy

Running one agent is easy. The hard part is everything else: can you build the infrastructure to run agents reliably at scale?

Running one agent is easy—and Oz's agent is best-in-class, highly ranked on benchmarks including Terminal-Bench and SWE-bench. But the agent itself isn't the bottleneck. The hard part is everything else: session sharing so your team sees what it did, cloud-to-local handoff so engineers can pick up where agents left off, observability for compliance, and integration maintenance as APIs and models evolve.

Remember that 75% failure rate for DIY agentic systems? The gap is orchestration, governance, and reliability—not the agent itself.

Oz time-to-value: You can have your first agent running in hours—simple workflows can be live in minutes. Team-wide rollout of complex, autonomous agents with governance takes weeks, not quarters. Compare that to DIY: weeks to build basic orchestration, months to reach production-ready, and then ongoing maintenance as the AI ecosystem evolves around you.

What you get with Oz that's hard to build:

- Session sharing attached to every PR—your team sees exactly what the agent did and why
- Cloud-to-local handoff—engineers continue where agents left off without losing context
- Full audit trail—every run, every decision, logged and searchable
- Continuous updates—as model providers evolve, APIs change, and new capabilities emerge

Building this yourself means dedicating engineers to agent infrastructure instead of product work—and maintaining it indefinitely.

Where Is Your Team?

AI Adoption Scale

Isolated

System-Level

Most organizations fall into one of three stages:

Ad-hoc AI adoption

- Engineers use AI tools in isolation (Claude, Cursor, Devin experiments)
- No visibility, no governance, no way to measure impact
- Individual productivity up, but doesn't compound across team

DIY orchestration

- Complex environment setup to give agents repository and tool access
- No org-wide standards, compliance gaps, growing maintenance burden
- Engineers spending time on agent infrastructure instead of product

Oz for orchestration

- Team-wide automation with full visibility and governance
- Agents run in background, across repos; and your code stays within your security boundary
- One person configures, entire team benefits

Why This Matters Now

AI-assisted development is no longer experimental—it's table stakes. The differentiator now is orchestration: which teams can scale agents beyond individual productivity into team-wide automation.

Teams that figure out agent orchestration in the next 12-18 months will compound their advantage. Every automated workflow frees engineering time for the next one. Every hour saved on incident triage or tech debt cleanup goes back into feature development.

Teams that wait will be catching up to competitors who automated their SDLC while they were still debating build vs. buy.

Your First Automated Workflow

Here's what the path to production looks like:

Typical adoption path (based on early customers)

- **Week 1:** Engineering team configures integrations; engineers start offloading existing agent runs to Oz cloud. Start from pre-built templates (PR review, issue triage, doc updates) or write your own.
- **Week 2:** Deploy first scheduled agent (e.g., documentation updater)—immediate reduction in team toil
- **Week 3:** Team sees value, deploys additional use cases
- **Week 4+:** Scale to multi-repo coordination and complex orchestration

What to look for when evaluating agent platforms

- **Security:** Deployment options, data retention, access controls
- **Scalability:** Multi-repo support, parallel execution
- **Developer experience:** Ease of agent definition, debugging, local-to-cloud parity
- **Compliance:** Audit logs, run history, cost controls
- **ROI:** Track via DORA metrics (lead time, deployment frequency, MTTR)

Start with what you have. Oz runs alongside your existing tools—no infrastructure change, no migration. Agents create PRs, so your review process stays the same. Start with a single use case, prove value, then expand. The risk is low; the upside compounds.

You're not doing this alone. Enterprise customers get hands-on implementation support—we'll help you configure your first workflows, troubleshoot integration issues, and identify the highest-ROI use cases for your team. This isn't generic onboarding; it's engineering support from people who've deployed agents at scale.

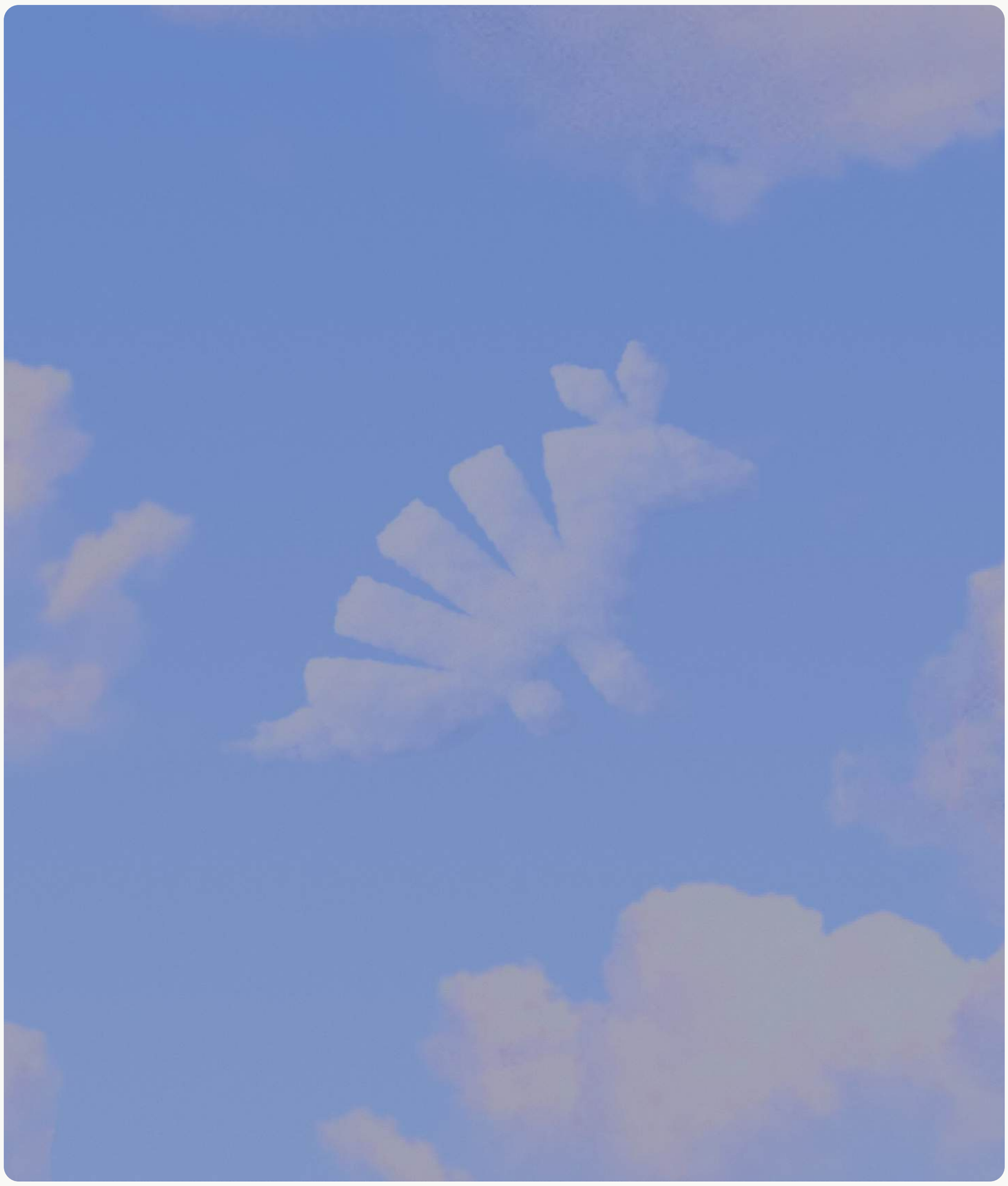
Schedule a 30-Minute Technical Discovery Call

Hands-on implementation planning with engineers who've deployed agents at scale.

- We'll map your current AI tool usage and identify the highest-value automation opportunity
- Walk away with a concrete recommendation and a plan to implement it
- Enterprise customers get dedicated support through first deployment—and we'll be honest if Oz isn't the right fit

Talk to an implementation engineer

Not ready for a call?

- Explore the docs: Visit our docs
- See Oz in action: View the demo
- Share this PDF with your engineering leadership to start the internal conversation

"It's the difference between giving engineers a faster bicycle and giving the organization a transportation system."